

**VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING**



REPORT

STREAMING LAKEHOUSE DATA PLATFORM

COURSE: DISTRIBUTED SYSTEMS

SUPERVISOR(s): PhD. NGUYEN QUANG HUNG

——o0o——

STUDENT 1: NGO QUANG HIEU - 2311015

STUDENT 2: <NAME> - 2311016

STUDENT 3: <NAME> - 2311017

HO CHI MINH CITY, Sep 2026

Declaration of Authenticity

We declare that we solely conducted this specialized project, under the supervision of PhD. Diep Thanh Dang at the Faculty of Computer Science and Engineering, Vietnam National University - Ho Chi Minh City University of Technology.

We have taken care to properly acknowledge and document all external sources and references used in the project.

If there is any instance of plagiarism, we are ready to accept the consequences. Ho Chi Minh City University of Technology - Vietnam National University HCMC will not be held responsible for any copyright violations that may have occurred during my research.

Ho Chi Minh City, December 2027

Authors,

Ngo Quang Hieu

Acknowledgement

We would like to express my appreciation to Dr. Diep Thanh Dang for his invaluable guidance. Our research has greatly benefited from his deep knowledge, perceptive criticism, and constant support.

Besides, we are grateful for the help from ... for valuable comments for research aspect and our proposed pipeline, including pointing out strengths as well as weaknesses of previous works on multimodal models so that we can elaborate on our ideas more effectively.

In addition, we would like to extend our gratitude to my family. Their unwavering faith in our abilities and constant encouragement have been my pillars of strength. Their belief in my potential has been a constant source of motivation and resilience. We are forever grateful for their love and support.

Abstract

This thesis presents *Streamify*, an end-to-end streaming lakehouse pipeline built to demonstrate real-time data engineering under a small, fixed compute budget. Synthetic music-streaming events, generated by Eventsim against the Million Song Dataset, are ingested through a 2-node Apache Kafka (KRaft) cluster, consumed by an Apache Spark Structured Streaming job, and landed as MERGE_ON_READ Apache Hudi bronze tables on MinIO/S3-compatible object storage.

dbt then transforms the bronze layer, through the Spark Thrift Server, into a COPY_ON_WRITE star schema (a fact_streams table and five dimension tables) and a denormalized wide_streams table, which Apache Superset visualizes through interactive dashboards. A Prometheus/Grafana/cAdvisor stack monitors Kafka consumer lag and Spark resource usage throughout, and the entire system is deployed reproducibly with a single Docker Compose command.

Every major technology choice in the pipeline – Kafka over HTTP or RabbitMQ, Spark Structured Streaming over Flink, Hudi over plain Parquet or a managed warehouse, dbt over hand-written SQL scripts, and Spark Standalone over YARN or Kubernetes – is justified against concrete alternatives and the resource constraints of a single-machine, 4-core deployment, rather than assumed as a given. This thesis documents that reasoning alongside the resulting architecture and its observed operational behavior.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Goals	2
1.3	Scope	3
1.4	Thesis Structure	3
2	Related Works	5
2.1	Alternative Technologies	5
2.1.1	Ingestion: Apache Kafka vs. Plain HTTP/REST vs. RabbitMQ	5
2.1.2	Kafka Cluster Sizing: 2-node KRaft with Replication Factor 2 .	6
2.1.3	Stream Processing: Apache Flink vs. Apache Spark Structured Streaming	7
2.1.4	Storage & Table Format: Apache Hudi vs. Plain Parquet, Delta Lake/Iceberg, and Snowflake	8
3	Theoretical Background	11
3.1	Core Theory and Concepts	11
3.1.1	Why Real-Time Processing Matters	11
3.1.2	Event Streaming and Publish-Subscribe Messaging	12
3.1.3	Spark Structured Streaming and Micro-batching	13
3.1.4	Table Formats: Apache Hudi (MERGE_ON_READ vs. COPY_ON_WRITE) 14	
3.1.5	Star-Schema Dimensional Modeling	15
4	Problem Statement and Challenges	16
4.1	Dataset	16

4.2	Technical Challenges	16
4.2.1	Streaming Volume and Velocity	16
4.2.2	Exactly-once, Fault-tolerant Processing on a 2-node Kafka Cluster	17
4.2.3	Computing and Partitioning on a Small, Shared Spark Cluster .	17
4.2.4	Dirty Keys Breaking Dimensional Joins	17
5	Proposed Solution	19
5.1	Architecture Overview	19
5.2	Ingestion Layer	21
5.3	Stream Processing Layer	21
5.4	Storage Layer	23
5.5	Transformation Layer	23
5.6	Serving Layer	25
5.7	Deployment & Environment	26
6	Evaluation and Experiments	27
6.1	Test Environment	27
6.2	Test Cases	27
6.3	Monitoring as Qualitative Evidence	28
6.4	Scalability	28
7	Conclusion	30
7.1	Results Achieved	30
7.2	Summary of Design Decisions	30
7.3	Limitations	32
7.4	Future Directions	32
	References	32

List of Tables

5.1	Technology stack per pipeline layer.	20
6.1	Scalability results (to be measured).	29
7.1	Summary of major design decisions and their deciding trade-offs. . . .	31

List of Figures

1	Streamify architecture overview: Eventsim, Kafka, Spark Structured Streaming, Hudi, dbt, Superset, and monitoring.	19
2	Star schema: fact_streams and its five dimension tables.	24
3	dbt DAG showing the dependency between bronze sources, the star schema, and wide_streams.	24
4	Superset results dashboard, built on top of wide_streams and queried through the Spark Thrift Server.	25
5	Grafana Kafka Lag dashboard: consumer-group lag and consumption rate per topic.	28
6	Grafana Spark dashboard: streaming/batch job metrics, JVM heap, and per-container CPU/memory.	28

Chapter 1

Introduction

In chapter 1, the overview, objectives, and goals of the research project are illustrated. The outline of the report is also presented.

1.1 Motivation

Modern data platforms increasingly need to react to events as they happen rather than waiting for the next batch load. A music-streaming service, for example, wants to know within seconds which songs are trending, whether a user’s session just dropped, or whether ingestion is falling behind – questions a traditional overnight ETL job cannot answer. Building and operating such a platform correctly requires more than a single tool: it requires a message queue that can buffer and replay events, a stream-processing engine that can keep up with continuous arrival, a storage layer that supports transactional updates on top of cheap object storage, a transformation layer that turns raw events into trustworthy analytics tables, and a way to observe the whole system’s health while it runs.

This project, *Streamify*, was motivated by the goal of building and demonstrating exactly such an end-to-end pipeline, using synthetic but realistic music-streaming events generated by Eventsim, rather than studying any single one of these components in isolation. Each stage of the pipeline – ingestion, stream processing, lakehouse storage, transformation, and visualization – was chosen and configured to work together as a single reproducible system, so that the trade-offs made at each stage (Chapter 2) could be

evaluated in the context of the whole pipeline rather than as isolated technology choices.

A further motivation was to explore these technologies under realistic resource constraints. Rather than assuming unlimited cloud compute, the project deliberately targets a small, single-machine deployment (Section 5.7), so that decisions such as Kafka cluster sizing and Spark cluster sizing (Sections 2.1.2 and 5.7) had to be made under the same resource pressure a small team or a proof-of-concept deployment would face in practice.

1.2 Goals

The goals of this project are:

- **Build a working, end-to-end streaming lakehouse pipeline.** Take synthetic music-streaming events from generation (Eventsim) through ingestion (Kafka), stream processing (Spark Structured Streaming), lakehouse storage (Hudi), transformation (dbt), and visualization (Superset), as a single reproducible system rather than isolated proofs of concept.
- **Demonstrate streaming-specific concerns, not just data movement.** Exercise partitioning, replication, checkpointed exactly-once processing, and consumer-lag monitoring, so the pipeline shows the behaviors that distinguish a streaming design from a periodic batch job (Section 3.1).
- **Justify every major technology choice against concrete alternatives.** For each stage of the pipeline, identify the alternatives that were available, the trade-off that mattered for this project, and why the chosen option won (Chapter 2), instead of treating the stack as a given.
- **Operate within a small, fixed-size compute budget.** Design the Kafka and Spark cluster sizing, and the choice of table format and transformation tooling, around a resource-constrained, single-machine deployment rather than assuming elastic cloud scale.
- **Make the whole system observable and reproducible.** Provide Prometheus/Grafana monitoring of Kafka lag and Spark resource usage, and deploy the entire stack with a single Docker Compose command.

1.3 Scope

This project covers the full path from synthetic event generation to dashboarding: Eventsim event generation, Kafka ingestion, Spark Structured Streaming processing into Hudi bronze tables, dbt transformation into a star schema and wide table, Superset dashboarding, and Prometheus/Grafana monitoring of the pipeline’s own health (Chapter 5). All of these components are deployed and demonstrated together via Docker Compose on a single machine.

The following are explicitly out of scope:

- **Production security hardening.** Authentication, authorization, network isolation, secrets management, and TLS between services are not addressed; the stack is configured for local development and demonstration, not for exposure to untrusted networks.
- **Multi-region or high-availability production deployment.** The pipeline targets a single-machine, single-region Docker Compose deployment (Section 5.7); cross-region replication, multi-datacenter failover, and elastic autoscaling of the Kafka or Spark clusters are not implemented.
- **Real user data.** All events are generated by Eventsim against the Million Song Dataset’s 10,000-song subset (Chapter 4) and are entirely synthetic; the pipeline has not been run against real production traffic or real user data.

1.4 Thesis Structure

The remainder of this report is organized as follows:

- **Chapter 2, Related Works,** reviews alternative technologies considered for each stage of the pipeline and the reasons for the choices made in this project.
- **Chapter 3, Theoretical Background,** covers the core concepts behind the pipeline: event streaming and pub-sub messaging, Spark Structured Streaming’s micro-batch model, Hudi’s MERGE_ON_READ and COPY_ON_WRITE table formats, and star-schema dimensional modeling.

- **Chapter 4, Problem Statement and Challenges**, describes the dataset used and the technical challenges of building a fault-tolerant streaming pipeline on a small, shared compute cluster.
- **Chapter 5, Proposed Solution**, presents the system architecture, the implementation workflow, and the cluster/environment setup.
- **Chapter 6, Evaluation and Experiments**, describes the test environment, test cases, and monitoring evidence gathered.
- **Chapter 7, Conclusion**, summarizes the results achieved, current limitations, and directions for future work.

Chapter 2

Related Works

2.1 Alternative Technologies

This chapter reviews the established, existing solutions for the four problems this pipeline must solve – event ingestion, cluster sizing, stream processing, and lakehouse storage – before comparing how this project’s own choices, described in Chapter 5, relate to that existing field. Each subsection below follows the same pattern: what already exists and is widely used in industry and research, which alternatives were weighed for this project, and why the chosen option fits here.

2.1.1 Ingestion: Apache Kafka vs. Plain HTTP/REST vs. RabbitMQ

Kafka’s log-based publish/subscribe model was originally built at LinkedIn to carry high-throughput activity-stream and operational-metrics data between systems, and its design was later published as the now widely-cited Kafka paper [3]. Since then, a durable, replayable log broker of this kind has become the de facto standard message queue at the ingestion boundary of streaming architectures industry-wide, underpinning both the Lambda and Kappa architecture patterns. Against that established backdrop, three ways of getting Eventsim’s generated events into this pipeline were considered.

- **Plain HTTP/REST ingestion.** Eventsim could post each event directly to an HTTP endpoint backed by the storage layer. This needs no separate broker, but

it couples producer and consumer directly in time: a burst of events forces the receiving endpoint to keep up synchronously, there is no built-in buffering, and there is no way to replay or reprocess events once they have been received.

- **RabbitMQ.** A classic message broker with per-message acknowledgment and flexible exchange/queue routing. Its model is built around dispatching a message to a queue and removing it once consumed; replaying history for a new or failed consumer generally requires the message still to be present in a queue, and running two independent consumers over the same events (e.g. the streaming job today, an ad-hoc reprocessing job later) is not the primary use case the queue model was designed for.
- **Apache Kafka [3].** A durable, append-only, partitioned log. Consumers track their own read position (offset) per topic and per consumer group, so the same retained log can be replayed and consumed independently by multiple readers without the broker deleting data on consumption.

The trade-off that mattered here was decoupling and replay: the pipeline needs Structured Streaming to resume exactly where it left off after a restart (Section 3.1), and needs the option to reprocess or backfill from the same event history without re-running Eventsim. Kafka’s retained, partitioned log with per-consumer-group offsets satisfies both directly, which is why it was chosen over both a direct HTTP push and a traditional message queue.

2.1.2 Kafka Cluster Sizing: 2-node KRaft with Replication Factor

2

While the previous subsection reviewed Kafka as a technology choice at industry scale, cluster sizing itself is a project-specific configuration decision rather than an industry-wide standard: broker count and replication factor are always set to match a deployment’s own fault-tolerance and resource budget. For this project, three cluster sizes were possible: a single broker, a two-broker cluster, or a larger cluster of three or more brokers.

- **Single broker.** Simplest to run, but every partition has exactly one copy: a broker

restart or crash makes its partitions unavailable until it comes back, with zero fault tolerance.

- **Larger cluster (3+ brokers, replication factor 3).** Higher fault tolerance, but needs a third (or more) broker container running on the same machine that already hosts the Spark cluster, Hudi/MinIO, dbt's Thrift Server, Superset, and the monitoring stack.
- **Two brokers, replication factor 2.** Every partition has a replica on both brokers, so the cluster keeps serving all partitions if either single broker restarts or fails, using only two broker containers.

Given the whole stack runs via Docker Compose on a single, resource-constrained machine where the Spark cluster alone already claims 4 cores (Section 5.7), replication factor 2 on two brokers was the smallest configuration providing any broker-level fault tolerance without paying for a third broker container – this is why the project settled on a 2-node KRaft cluster with replication factor 2 rather than a single broker or a larger cluster.

2.1.3 Stream Processing: Apache Flink vs. Apache Spark Structured Streaming

Apache Flink and Apache Spark Structured Streaming are the two dominant distributed stream-processing engines used in industry and research today, and this project's own pipeline evaluated both directly. Apache Flink [1] is built around native, event-at-a-time streaming with low end-to-end latency, and is the engine of choice for organizations whose workloads demand very low end-to-end latency; an earlier version of this project used a Flink SQL job for the Kafka-to-lakehouse ingestion path. Apache Spark Structured Streaming [5], in contrast, models streaming as a sequence of small batch jobs (micro-batching) on top of the same engine used for batch and SQL workloads, and is widely adopted where a single engine needs to serve both batch and streaming workloads.

The project replaced the Flink job with a Spark Structured Streaming job (Section 3.1) for two practical reasons:

- **Operational simplicity and unified engine.** Running Flink alongside Spark means operating two separate distributed runtimes – each with its own cluster manager, checkpointing/savepoint mechanism, and deployment model – on a cluster with only 4 cores in total (Section 5.7). dbt already needed a Spark cluster (via the Spark Thrift Server) for the batch transformation layer, and the team’s existing familiarity was with Spark’s DataFrame/SQL API from that batch layer. Consolidating stream and batch processing onto one engine and one operational model reduces the surface area a small team has to maintain, at the cost of giving up Flink’s lower-latency, event-at-a-time execution.
- **Native Hudi integration.** Apache Hudi ships first-class Spark data source support, which simplifies writing MERGE_ON_READ bronze tables directly from the streaming job without an extra connector layer.

The trade-off is latency: Flink’s native streaming model can achieve lower per-event latency than Spark’s micro-batch model, since Spark’s output only becomes visible once a whole micro-batch trigger interval has elapsed. For this project’s throughput and cluster size, operational simplicity and team familiarity with a single, already-shared engine outweighed the latency advantage that a separate, lower-latency Flink cluster would have provided.

2.1.4 Storage & Table Format: Apache Hudi vs. Plain Parquet, Delta Lake/Iceberg, and Snowflake

Getting ACID transactions and row-level upsert/delete semantics on top of plain files in a data lake is a problem the broader industry has already converged on a category of solution for: the open table format, with Apache Hudi, Delta Lake, and Apache Iceberg established as the three dominant open table formats in current use, alongside proprietary managed warehouses such as Snowflake that solve the same problem by fully owning storage and compute. Four options along this spectrum were considered for how the lakehouse layer stores and manages tables.

- **Plain Parquet files, no table format layer.** Cheapest to write, but plain Parquet has no row-level upsert/delete semantics, no ACID commits, and no time travel; a

separate catalog service (such as a Hive Metastore or a cloud catalog) would still be needed to track partitions and schema, adding another piece of infrastructure to operate.

- **Delta Lake / Apache Iceberg.** Both are comparable open table formats that add ACID transactions and schema evolution on top of object storage, similar in spirit to Hudi.
- **Snowflake [7], a cloud data warehouse.** Fully managed, with storage and compute separated, but it couples the project to a proprietary, hosted platform and its billing model.
- **Apache Hudi [2] (chosen).** A transactional table format for data lakes with row-level insert/update/delete semantics, ACID commits, and time travel over plain files on object storage.

Two concrete reasons drove the choice of Hudi specifically:

- **No separate catalog service.** Each Hudi table is self-describing: it carries its own `.hoodie/` metadata directory at its storage path, and Spark (re-)registers the table into its own catalog idempotently at job/run startup. This avoids standing up and operating a separate catalog service (as plain Parquet would require) purely to track table metadata, which matters for a project meant to be deployable end-to-end with a single Docker Compose command.
- **Explicit `MERGE_ON_READ` / `COPY_ON_WRITE` split matching the pipeline's two write patterns.** Hudi's distinction between MOR and COW table types (Section 3.1) maps directly onto the pipeline's two different write patterns: frequent, small streaming appends into the bronze layer, and full-overwrite batch rebuilds into the gold layer. Bronze tables use MOR because the streaming job appends small batches of new events continuously; under COW, every micro-batch write would rewrite entire Parquet files, multiplying write amplification with append frequency. MOR instead appends cheaply to a compact row-based delta log, deferring the cost of producing a fully merged, columnar view to a background compaction job. Gold tables use COW instead, because dbt rebuilds them via full overwrite

once per run and Superset then reads them repeatedly for dashboards: COW pays the rewrite cost once per run, after which every dashboard query is a plain Parquet scan with no read-time merge step, whereas MOR would push that merge cost onto every single read.

Compared to Snowflake, adopting the lakehouse pattern (raw and modelled data as Hudi tables on MinIO locally, or Amazon S3 in aws mode via the Hadoop S3A connector, with Spark/dbt querying that storage directly) keeps the storage layer open-format and keeps the whole stack deployable on a single machine or small cluster with Docker Compose, which suited this project's development and demonstration constraints better than committing to a managed cloud warehouse. Compared to Delta Lake or Iceberg, Hudi's explicit MOR/COW split was the deciding factor, since it lets the bronze and gold layers each use the write/read profile that matches their actual access pattern instead of a single table type across both.

Chapter 3

Theoretical Background

3.1 Core Theory and Concepts

This section summarizes the underlying concepts of the technologies used to build the pipeline in Chapter 5.

3.1.1 Why Real-Time Processing Matters

A modern platform such as a music-streaming service does not produce data in daily batches: it produces a continuous stream of small events – a song played, a page viewed, a login – the instant a user acts. Streamify, the project this report builds, simulates exactly this kind of platform with Eventsim, which generates synthetic “song played”, “page view”, and “authentication” events for a fake music website at a steady, ongoing rate.

In a traditional batch-processing setup, these events would simply accumulate in a source system and be pulled into the warehouse on a fixed schedule – once an hour, once a day. Between runs, every dashboard and every downstream decision is working from data that is, on average, half of that interval out of date; right before the next run, it is a full interval out of date. For a platform that wants to react to what users are doing now – surfacing trending songs, flagging a spike in failed logins, feeding a recommendation engine – that lag is the problem: the data is correct, just always late.

Real-time (streaming) processing closes this gap. Instead of waiting for a scheduled run, each event is picked up and made queryable within seconds to minutes of happening,

so dashboards and downstream models reflect what is happening now rather than what happened last interval. That said, doing this reliably, and not just quickly, is harder than it sounds once the platform has more than one producer and more than one consumer of the same data: events from different partitions can arrive out of order, any component in the chain can crash mid-stream, the same event must not be silently duplicated or dropped on retry, upstream fields will change shape over time, and the storage layer has to serve both a constant stream of small writes and large analytical reads without one starving the other. These are the distributed-systems problems that ordering guarantees, replication, exactly-once processing, and lakehouse table formats exist to solve, and they are exactly the concepts the rest of this chapter introduces: Apache Kafka for ordered, fault-tolerant ingestion, Spark Structured Streaming for continuous, exactly-once processing, and Apache Hudi for a storage layer that supports both frequent streaming writes and fast analytical reads.

3.1.2 Event Streaming and Publish-Subscribe Messaging

Event streaming treats data as an unbounded sequence of immutable events (e.g. “user played a song”) rather than as static rows in a table. Producers append events to a log; one or more consumers read that log independently and at their own pace. Apache Kafka [3] implements this as a publish-subscribe (pub-sub) system: producers publish events to named *topics*, and consumers subscribe to the topics they need, decoupling producers from consumers in time and in number.

Partitioning and Replication

A Kafka topic is split into *partitions*, each an ordered, append-only log. Partitioning is Kafka’s unit of parallelism: different partitions of the same topic can be produced to and consumed from concurrently, and ordering is only guaranteed within a single partition. Each partition can additionally be *replicated* across multiple brokers (replication factor); one replica is the leader that serves reads and writes, and the others are followers that copy the leader’s log. If a broker holding the leader replica fails, a follower on a surviving broker can take over, so the topic keeps serving as long as at least one replica of each partition survives.

KRaft vs. ZooKeeper

Earlier Kafka versions relied on Apache ZooKeeper, a separate coordination service, to elect partition leaders and store cluster metadata. KRaft (Kafka Raft) replaces that external dependency with a Raft-based consensus protocol run by the Kafka brokers themselves, so the brokers agree on cluster metadata directly instead of through a second distributed system. This removes an entire service (and its own quorum, ports, and failure modes) from the deployment, which matters for a small, resource-constrained cluster where every additional coordination process competes for the same limited memory and CPU.

Consumer Groups and Offsets

Kafka consumers read as part of a named *consumer group*: Kafka assigns each partition of a subscribed topic to exactly one consumer within the group, so the group as a whole divides the work of consuming a topic. Each consumer tracks its *offset* – the position of the last record it has read – per partition. Because a consumer’s progress is expressed purely as an offset, a consumer (or its replacement) can resume from exactly where it left off after a restart, rather than re-reading the whole topic or losing its place, as long as the offset is durably recorded somewhere.

3.1.3 Spark Structured Streaming and Micro-batching

Apache Spark Structured Streaming [5] runs a streaming query as a series of small, discrete batch jobs (*micro-batches*) on Spark’s existing batch execution engine, rather than processing each event individually. On a fixed trigger interval, the engine reads whatever new data has arrived since the last batch (tracked via checkpointed offsets), applies the same DataFrame/SQL transformations used for batch queries, and writes the result. This design reuses Spark’s mature batch engine (query planning, fault tolerance, exactly-once output via checkpointing) at the cost of end-to-end latency being bounded by the micro-batch interval rather than being event-at-a-time.

Checkpointing and Exactly-once Output

Structured Streaming persists two things to a durable checkpoint location after each micro-batch: which input offsets that batch covered, and the state needed to avoid re-processing them. On restart, the engine reads the checkpoint to determine exactly which offsets have already been committed to the output, and resumes from there. Combined with an idempotent or transactional sink, this gives *exactly-once* output semantics – each input record affects the output exactly once, even across job restarts – without the engine needing to coordinate with the source system beyond reading its offsets.

3.1.4 Table Formats: Apache Hudi (MERGE_ON_READ vs. COPY_ON_WRITE)

Apache Hudi [2] is a transactional table format for data lakes: it adds row-level insert/update/delete semantics, ACID commits, and time-travel to plain files on object storage (S3, MinIO, HDFS, ...). Hudi supports two table types that trade off write cost against read cost:

- **COPY_ON_WRITE (COW).** Every write rewrites the affected data files as new versioned Parquet files. Reads are fast (plain Parquet, no merge step), but writes are more expensive because entire files are rewritten on update.
- **MERGE_ON_READ (MOR).** Writes are appended to a compact row-based delta log next to the existing base Parquet files; the two are merged only at read time (or during a background compaction job). Writes are cheap and low-latency, at the cost of extra work on read (or a periodic compaction step) to materialize the merged view.

MOR suits high-frequency streaming appends, where write latency matters most; COW suits tables that are rebuilt in full on a schedule and then read many times, where read performance matters most.

Medallion Layering: Bronze and Gold

The medallion architecture organizes a lakehouse into successive layers of increasing refinement, conventionally named bronze, silver, and gold. *Bronze* tables hold raw or

lightly-cleaned data close to its source form, written as-is by the ingestion layer. *Gold* tables hold data that has been modeled and aggregated into the shape consumers (BI tools, dashboards) actually query, typically built by a transformation layer reading from bronze. This project uses the bronze and gold terms in that sense throughout: bronze is what Spark Structured Streaming writes directly from Kafka, and gold is what dbt builds on top of bronze.

3.1.5 Star-Schema Dimensional Modeling

Dimensional modeling organizes an analytical warehouse around a central *fact table* of measurable events (here, one row per stream/play event) surrounded by *dimension tables* of descriptive attributes (who, what, where, when) that the fact table references by key. This “star” shape – one fact table joined to several dimension tables – normalizes descriptive attributes into reusable dimensions while keeping the fact table narrow, and is the standard layout for BI and reporting workloads because it keeps common queries (aggregate the fact table, filter/group by a dimension) to a small, predictable number of joins.

Slowly Changing Dimensions (Type 2)

A dimension’s attributes can change over time – e.g. a user upgrading from a free to a paid subscription level – and a *Slowly Changing Dimension Type 2* keeps that history instead of overwriting it. Each change is stored as a new row with a validity window (an activation date and an expiration date), and the fact table joins to whichever dimension row was valid at the time of the event, rather than to a single current row. This preserves the ability to ask what a dimension’s attributes were at any past point in time, not just what they are now.

Chapter 4

Problem Statement and Challenges

4.1 Dataset

The pipeline is driven by Eventsim, a synthetic event generator that simulates page-request and listening activity on a fake music-streaming website (the Docker image used is viirya’s fork of the original, unmaintained project). Eventsim draws its song catalog from the Million Song Dataset; this project uses the official 10,000-song subset. The generated events (`listen_events`, `page_view_events`, `auth_events`) look like real user-activity logs but are entirely synthetic, which makes them safe to publish and reproduce without any user-privacy concern.

4.2 Technical Challenges

Building a streaming lakehouse pipeline on top of this dataset raised the following challenges.

4.2.1 Streaming Volume and Velocity

Eventsim continuously emits events across three topics, so the pipeline must ingest, decode, and land data in the lakehouse without falling behind the producer. This requires a message queue that can buffer and parallelize consumption (Kafka, partitioned per topic) and a streaming engine that can keep its micro-batch processing time below the

arrival rate.

4.2.2 Exactly-once, Fault-tolerant Processing on a 2-node Kafka Cluster

Kafka runs as a 2-node KRaft cluster (no ZooKeeper), with each topic replicated across both brokers (replication factor 2, 3 partitions per topic). With only two brokers, the cluster must keep serving every partition if either single broker is restarted or fails, while Spark Structured Streaming must resume exactly where it left off after any restart. This project relies on Structured Streaming's checkpointed offsets (persisted to a durable checkpoint volume) as the source of truth for resuming the stream, and on Kafka's per-partition replication across the two brokers for broker-level fault tolerance.

4.2.3 Computing and Partitioning on a Small, Shared Spark Cluster

The Spark Standalone cluster totals only 4 cores across one master and two workers (2 cores / 2 GB each), and those 4 cores are shared between two long-running applications: the Spark Structured Streaming ingestion job (capped at 2 cores) and the Spark Thrift Server that dbt submits its SQL transformations through (also capped at 2 cores). This leaves no spare capacity for ad-hoc Spark SQL at this cluster size, and means both the streaming job's micro-batch partitioning and dbt's batch transformation jobs must fit within their fixed 2-core budget rather than being able to scale out elastically.

4.2.4 Dirty Keys Breaking Dimensional Joins

Eventsim's raw event fields are not clean enough to join on directly. Text fields such as `artist` and `song` arrive with latin1-mojibake encoding, which the streaming job corrects before the bronze tables are written. Even after that correction, the same artist and song names carry stray quote and backslash characters that differ inconsistently between the `listen_events` stream and the artist/song dimension source, so `fact_streams` cannot join on the raw strings: it strips quotes and backslashes from both sides of the join (`REPLACE(REPLACE(artist, '"', ''), '\\', '')`) before matching against `dim_artists` and `dim_songs`. This is a concrete instance of a broader challenge for

the pipeline: dimensional joins are only as reliable as the string normalization applied to their join keys, and that normalization has to be decided and implemented explicitly rather than assumed.

Chapter 5

Proposed Solution

5.1 Architecture Overview

Figure 1 shows the end-to-end architecture: Eventsim generates events into a Kafka cluster, Spark Structured Streaming consumes and lands them as Hudi bronze tables, dbt transforms the bronze layer into a star schema and a wide table on the same Spark cluster, and Superset visualizes the result through the Spark Thrift Server, alongside a Prometheus/Grafana monitoring stack.

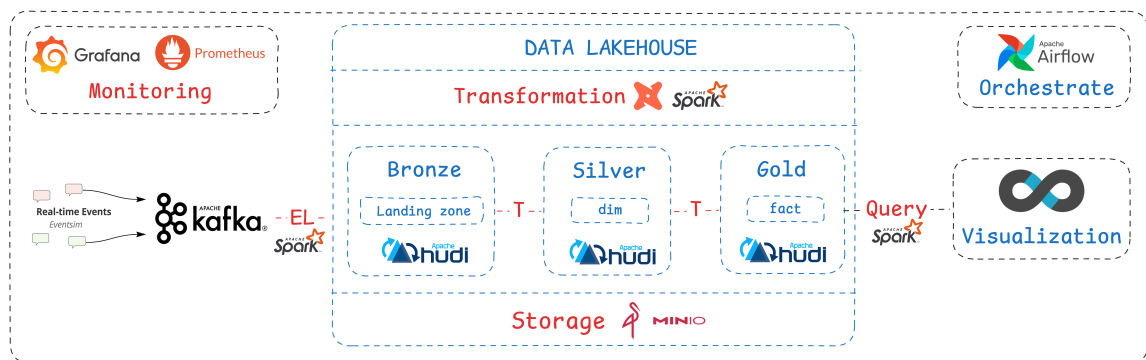


Figure 1: Streamify architecture overview: Eventsim, Kafka, Spark Structured Streaming, Hudi, dbt, Superset, and monitoring.

Table 5.1: Technology stack per pipeline layer.

Layer	Tool	Role
Data simulation	Eventsim	Generates synthetic music-streaming events based on the Million Song Dataset
Message queue	Apache Kafka 4.3 (KRaft)	2-node cluster, replication factor 2, 3 partitions per topic, no ZooKeeper
Stream processing	Spark Structured Streaming 3.5.3	Reads Kafka, decodes/enriches events, writes bronze tables to Hudi
Table format / storage	Apache Hudi	MERGE_ON_READ bronze streaming tables, COPY_ON_WRITE gold/seed tables, on MinIO/S3 via the Hadoop S3A connector
Compute cluster	Spark Standalone (1 master, 2 workers)	Shared by the streaming job and dbt (via Thrift), 4 cores total
Transformation	dbt (dbt-spark, method: thrift)	SQL-based star schema and wide-table modelling, submitted via

The remainder of this chapter walks through the pipeline one layer at a time – ingestion, stream processing, storage, transformation, and serving – followed by how the whole stack is deployed. Each layer section closes with a short note on why that specific tool was chosen, cross-referencing the fuller alternatives-and-trade-offs discussion in Chapter 2.

5.2 Ingestion Layer

Eventsim simulates a real-world user base, generating page-view and song-play events that mimic activity on a music-streaming website. Apache Kafka 4.3 runs in KRaft mode (no ZooKeeper) as a 2-node cluster (broker-1, broker-2). Each of the three event topics – `listen_events`, `page_view_events`, `auth_events` – has 3 partitions and replication factor 2, so every partition has a replica on both brokers.

Why Kafka, and why this sizing. Kafka’s durable, partitioned log and per-consumer-group offsets let Structured Streaming resume ingestion exactly where it left off after a restart, which a direct HTTP push or a traditional queue such as RabbitMQ does not provide as directly (Section 2.1.1). The 2-node, replication-factor-2 sizing is the smallest configuration that gives the cluster any broker-level fault tolerance at all, chosen over a single broker (no fault tolerance) or a larger cluster (more resource cost than this project’s single-machine deployment justifies) (Section 2.1.2).

5.3 Stream Processing Layer

A long-running Spark Structured Streaming job consumes all three Kafka topics, fixes latin1-mojibake on text fields, derives time columns, and appends each micro-batch to bronze Hudi tables (`staging.<topic>`).

Listing 5.1 shows the core of the job that reads a Kafka topic, decodes it, and derives the time columns later used to partition the bronze Hudi tables.

Listing 5.1: Building a Hudi bronze stream from a Kafka topic (excerpt from `spark_streaming/jobs/stream_events.py`).

```
1 def build_stream(spark, topic, decode_strings):  
2     raw = (
```

```

3      spark.readStream.format("kafka")
4      .option("kafka.bootstrap.servers", KAFKA_BROKERS)
5      .option("subscribe", topic)
6      .option("startingOffsets", "earliest")
7      .load()
8  )
9
10     parsed = raw.select(
11         F.from_json(F.col("value").cast("string"), SCHEMAS[topic]).alias
12         "partition",
13         "offset",
14     ).select("data.*", "partition", "offset")
15
16     if decode_strings:
17         parsed = (
18             parsed.withColumn("artist", decode_udf(F.col("artist")))
19             .withColumn("song", decode_udf(F.col("song")))
20         )
21
22     ts_col = F.timestamp_millis(F.col("ts"))
23     return (
24         parsed
25         .withColumn("ts_timestamp", ts_col)
26         .withColumn("year_", F.year(ts_col))
27         .withColumn("month_", F.month(ts_col))
28         .withColumn("day_", F.dayofmonth(ts_col))
29         .withColumn("hour_", F.hour(ts_col))
30         .withColumn("processed_time", F.current_timestamp())
31         .withColumn("_row_key", F.expr("uuid()"))
32     )

```

Structured Streaming resumes from a persisted checkpoint volume (spark-checkpoints) on restart; each micro-batch also commits its Kafka offsets to a streamify-`<topic>` consumer group purely so lag is visible in Grafana (Spark itself never depends on that

commit to resume).

Why Spark Structured Streaming, and not Flink. An earlier version of this project used Apache Flink for this job. It was replaced with Spark Structured Streaming so that stream processing runs on the same engine and cluster that dbt already needs for batch transformation (avoiding operating two separate distributed runtimes on a 4-core cluster), and to reuse Hudi's first-class Spark data source support. The trade-off accepted is latency: Flink's event-at-a-time model can achieve lower per-event latency than Spark's micro-batch model (Section 2.1.3).

5.4 Storage Layer

Tables are Hudi-managed, backed by MinIO/S3 locally or real S3 in aws mode via the Hadoop S3A connector, with no rebuild needed to switch between the two – only an environment variable (LAKE_ENV). There is no separate catalog service: each table is self-describing at its storage path (a `.hoodie/` metadata directory) and is (re-)registered into Spark's catalog idempotently at job startup. Bronze tables use the `MERGE_ON_READ` table type, so fast streaming appends land as delta log files that are periodically compacted into columnar base files; gold and seed tables use `COPY_ON_WRITE`, rebuilt via full overwrite.

Why Hudi, and why MOR bronze / COW gold. Hudi's self-describing `.hoodie/` metadata avoids the extra catalog or warehouse service that plain Parquet or Snowflake would require. Bronze uses MOR to keep frequent streaming appends cheap; gold uses COW because it is rebuilt once per run and then read repeatedly by Superset (Section 2.1.4).

5.5 Transformation Layer

dbt [6] connects to the Spark cluster via the Spark Thrift Server (method: `thrift`) and builds two model layers directly on top of the bronze Hudi tables, materialized as `COPY_ON_WRITE` Hudi tables rebuilt via full overwrite each run:

- **Star schema.** One fact table, `fact_streams`, joined to five dimension tables:

dim_users, dim_songs, dim_artists, dim_locations, and dim_datetime. This is the normalized, storage-efficient model (Section 3.1).

- **Wide table (wide_streams).** A denormalized table built from the star schema by pre-joining fact_streams with all five dimension tables, so that Superset can query a single flat table instead of performing joins at dashboard-query time.



Figure 2: Star schema: fact_streams and its five dimension tables.



Figure 3: dbt DAG showing the dependency between bronze sources, the star schema, and wide_streams.

Why dbt, and why both a star schema and a wide table. dbt was chosen over a layer of hand-written Spark SQL scripts because it declares model dependencies explicitly (via `ref()`/`source()`), runs a built-in test framework against every model (uniqueness, not-null, referential integrity), and generates documentation from the same source files – properties that matter given the five-dimension join the fact table depends on. Modelling both a star schema and a separate wide table, rather than either alone, keeps normalization and storage efficiency in the star schema while keeping BI query simplicity in `wide_streams`, instead of trading one off against the other.

5.6 Serving Layer

Apache Superset [4] connects to the same Spark Thrift Server that dbt submits SQL through, querying the `wide_streams` model to visualize insights through interactive dashboards (`superset/provision_dashboard.py` provisions them via the REST API). Figure 4 shows the resulting dashboard layout.

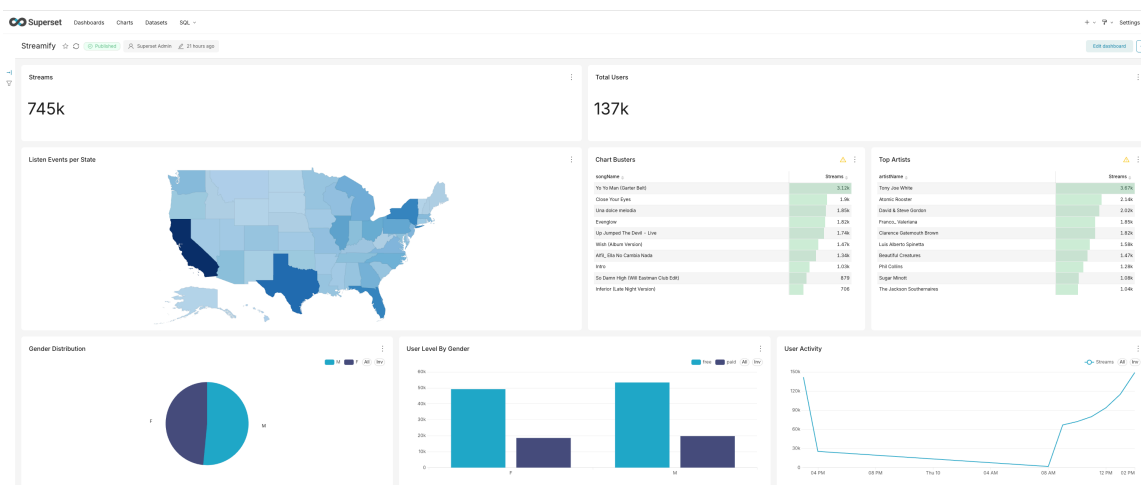


Figure 4: Superset results dashboard, built on top of `wide_streams` and queried through the Spark Thrift Server.

Prometheus scrapes Kafka consumer-group lag (via `kafka-exporter`), Spark’s built-in Prometheus metrics, and per-container stats (via `cAdvisor`); Grafana visualizes all of it across dashboards.

Why Superset. Superset ships a native Thrift/HiveServer2-compatible connector that talks directly to the Spark Thrift Server endpoint already used by dbt, needing no

extra license or gateway infrastructure, unlike Power BI or Tableau.

5.7 Deployment & Environment

The whole stack is deployed with Docker Compose (`docker-compose.yml`), with a `docker-compose.dev.yml` overlay that adds MinIO for local, S3-free development. Spark runs as a Standalone cluster of 1 master and 2 workers, 2 cores / 2 GB each (4 cores total), shared between the streaming ingestion job and the Spark Thrift Server that dbt submits SQL through, each capped at 2 cores.

Why Spark Standalone at this size. Spark Standalone was chosen over YARN or Kubernetes because both alternatives add a separate resource-management control plane that is not justified when the entire compute budget is 4 cores across one master and two workers; Standalone instead runs as three containers alongside the rest of the pipeline in the same `docker-compose.yml`. The direct consequence is that the streaming job and the Thrift Server, each capped at a fixed 2 cores, together already consume the entire cluster, leaving no headroom for ad-hoc Spark SQL while both are running.

Chapter 6

Evaluation and Experiments

6.1 Test Environment

The pipeline was exercised on the Docker Compose stack described in Section 5 (Kafka 2-node cluster, Spark Standalone with 1 master and 2 workers, MinIO for local object storage), run on a single local machine.

6.2 Test Cases

Two categories of tests were used to validate the pipeline:

- **Correctness, on a small dataset.** A small batch of Eventsim events is run through the full path – Kafka, Spark Structured Streaming, Hudi bronze, dbt star schema and wide table – and the resulting rows in `fact_streams` / `wide_streams` are checked against the source events for row counts, key fields, and referential integrity against the dimension tables.
- **Runtime and throughput, on a larger volume.** Eventsim is left running for an extended period to build up a larger event volume, and the streaming job’s micro-batch processing time and Kafka consumer lag are observed under sustained load to check that the pipeline keeps up with the producer on the available 4-core cluster.

6.3 Monitoring as Qualitative Evidence

The Grafana dashboards backed by Prometheus provide qualitative evidence of the pipeline’s throughput and health under load, rather than a formal benchmark.

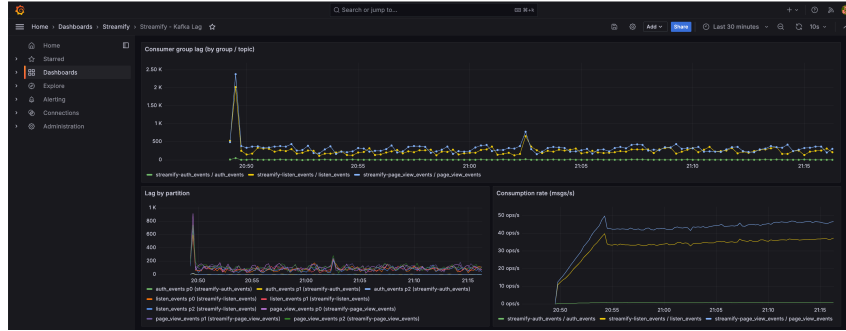


Figure 5: Grafana Kafka Lag dashboard: consumer-group lag and consumption rate per topic.



Figure 6: Grafana Spark dashboard: streaming/batch job metrics, JVM heap, and per-container CPU/memory.

6.4 Scalability

Table 6.1 is a placeholder for measured scalability results (event volume vs. Kafka/Spark cluster sizing vs. runtime). No such measurements exist in this repository at the time of writing (results/ is not present), so no numbers are reported here to avoid fabricating results.

Table 6.1: Scalability results (to be measured).

Event volume	Kafka brokers	Spark cores	Runtime

Chapter 7

Conclusion

7.1 Results Achieved

This project delivered a working, end-to-end streaming lakehouse pipeline: Eventsim-generated events flow through a 2-node Kafka cluster into a Spark Structured Streaming job that lands MERGE_ON_READ Hudi bronze tables, which dbt transforms (via the Spark Thrift Server) into a COPY_ON_WRITE star schema (fact_streams and five dimension tables) plus a denormalized wide_streams table. Apache Superset dashboards query that wide table through the same Thrift Server, and a Prometheus/Grafana/cAdvisor stack monitors Kafka consumer lag and Spark resource usage throughout. The entire stack is reproducible with a single Docker Compose command.

7.2 Summary of Design Decisions

Table 7.1 summarizes the major technology choices made throughout this project, the alternatives considered for each, and the trade-off that decided the outcome. The ingestion, stream-processing, and storage/table-format choices are discussed in full in Chapter 2; the remaining choices are discussed alongside the layer they belong to in Chapter 5.

Table 7.1: Summary of major design decisions and their deciding trade-offs.

Decision	Alternatives considered	Deciding trade-off
Kafka for ingestion	HTTP/REST push, RabbitMQ	Durable, re-playable log with per-consumer-group off-sets vs. no built-in re-play (Section 2.1.1)
2-node KRaft, RF2	Single broker, larger cluster	Smallest sizing giving any broker fault tol-er-

7.3 Limitations

- **Small, fixed-size cluster.** The Spark Standalone cluster totals only 4 cores, shared between the streaming job and the Thrift Server that dbt uses, leaving no headroom for ad-hoc Spark SQL or additional concurrent workloads.
- **No autoscaling.** Kafka brokers, Spark workers, and other services are fixed at deployment time; the pipeline does not scale resources up or down in response to load.
- **Synthetic dataset.** Eventsim generates realistic-looking but entirely synthetic user activity over a fixed 10,000-song catalog, so the pipeline has not been validated against real production traffic patterns or data-quality issues.

7.4 Future Directions

- Replace or augment the synthetic Eventsim data with a real or real-scale streaming dataset to validate the pipeline under realistic data-quality and volume conditions.
- Add autoscaling for the Kafka and Spark clusters (e.g. via a container orchestrator) so cluster capacity can grow with load instead of being fixed at deployment time.
- Extend monitoring and alerting beyond the current Grafana dashboards, e.g. automated alerts on consumer lag thresholds, streaming job failures, or dbt run failures.

References

- [1] Apache Software Foundation. *Apache Flink Documentation*. 2024. URL: <https://nightlies.apache.org/flink/flink-docs-stable/> (visited on 09/12/2026).
- [2] Apache Software Foundation. *Apache Hudi Documentation*. 2024. URL: <https://hudi.apache.org/docs/overview> (visited on 09/12/2026).
- [3] Apache Software Foundation. *Apache Kafka Documentation*. 2024. URL: <https://kafka.apache.org/documentation/> (visited on 09/12/2026).
- [4] Apache Software Foundation. *Apache Superset Documentation*. 2024. URL: <https://superset.apache.org/docs/intro> (visited on 09/12/2026).
- [5] Apache Software Foundation. *Structured Streaming Programming Guide*. 2024. URL: <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html> (visited on 09/12/2026).
- [6] dbt Labs. *dbt Documentation*. 2024. URL: <https://docs.getdbt.com/> (visited on 09/12/2026).
- [7] Snowflake Inc. *Snowflake Documentation*. 2024. URL: <https://docs.snowflake.com/> (visited on 09/12/2026).